How SD-JWT and ACDC are similar and different

Daniel Hardman

2024-11-15

Abstract

This article compares SD-JWT and ACDC technologies, discussing their similarities, differences, and implications for verifiable credentials.

Provenant is advocating that telco evidence be built from ACDCs, not SD-JWTs. How are these two technologies similar, how are they different, and why does it matter?

> Disclaimer: this information embodies what I believe to be true based on study and personal experience with the technologies. I see virtues in both technologies. I have consulted to help an SD-JWT-based initiative and I am actively involved in several ACDC-based projects at my day job. I am attempting to be fair, and I am happy to update what I say here if anybody wants me to correct an inaccuracy. However, I am more enthused about ACDCs, so I am not a perfectly unbiased source.

1. Similarities

SD-JWTs and ACDCs are both signed, serialized data structures that are roughly understandable with just the intuition of programmers who know JSON. Both support a compact form that is less human-friendly.

Both technologies were invented in the decentralized identity community. ACDCs were being standardized at Trust Over IP Foundation by early 2021. The first draft of SD-JWT appeared in IETF's datatracker in June 2022. A derivative RFC draft, SD-JWT-VC, adapts SD-JWT to W3C's Verifiable Credential data model. Its first draft was submitted in mid 2023 during a wave of effort on OIDC4VC.

The authors of these specs know one another, and I know both groups. They share ideas in community events like the IIW conference. It is not clear whether SD-JWTs derive from ACDCs, or are a wholly independent invention, or both share a common ancestor, or some combination of influence occurred. Both show thinking from Merkle-based data structures popularized in blockchain/cryptocurrency circles.

Both use a technique where salted hashes of credential attributes can replace plaintext attributes, allowing part of a credential to be shared and part to be hidden from the verifier. SD-JWT calls this feature "selective disclosure", and ACDCs calls it "graduated disclosure." Both make this feature optional.

> Note: The term "selective disclosure" has a long and unpleasant history in financial regulation, where it refers to the illegal behavior of a business leader who discloses

some financial details to one audience, and some to another, to perpetrate fraud. The decentralized identity community began using this term (unwisely, IMO) to describe a weaker alternative to the unlinkability of AnonCreds, around 2019. SD-JWTs built on the term. I prefer the ACDC term, because it won't confuse and put off people in the regulatory community.

2. Differences

2.1 Scope

Both technologies can create digital credentials (signed artifacts that confer entitlements on a holder), and both can support a more flexible mode where there is no provable binding to a holder. However, in practice, nearly all JWT usage in industry today targets proof of entitlement, the SD-JWT spec mentions verifiable credentials in its opening sentence, and SD-JWT implementer behavior focuses there. For evidence that isn't entitlement-oriented, (e.g., verifiable citations of data in science, journalism, or regulatory/legal ecosystems), ACDCs may be a more natural fit.

2.2 Identifiers

SD-JWTs allow an issuer and a holder to be identified with a DID, a URL, a URN, or some other identifier scheme. ACDCs are stricter: issuers MUST be identified with an AID, and issuees, if present, MUST also be identified with an AID.

> Note: AIDs (autonomic identifiers) are a KERI construct. They provide numerous security guarantees that are not true of identifier schemes in general, or of popular DID schemes, in particular. These include versioned and anchored key state, witnesses, prerotation, weighted multisig, postquantum migration path, self-certification, etc. AIDs can be transformed to DIDs, but the opposite transformation is typically impossible.

Flexibility in this dimension is a tradeoff. On one hand, it lets parties with many different security approaches, and many different ways to reference participants, use the same data structure; on the other, it makes it much more difficult to predict or enforce security properties, and it makes tooling less useful because theoretical and practical interop diverge.

2.3 Signing mechanism

SD-JWTs are signed by a key of the issuer. The key is often the same as the one in the issuer's X509 cert, which encumbers JWTs with some logistical and governance baggage (e.g., to track revocations). The key is typically referenced by URL or name in the `kid` field, and the signature is carried in a JWS, which is the third and final section of the JWT data structure.

The timing associated with the signature is self-asserted (via the `iat` claim). Since most JWTs are designed to expire (and thus, be verified) quickly, verifiers can compare `iat` to their system clock and discount anything that's too old. This minimizes the mischief that's possible with fraudulent dates. Retrograde attacks become unlikely, but features are lost: *tokens are only designed to be evaluated against current key state*. If keys change, all existing tokens become invalid. If you are asking a historical/audit-oriented question like, "Was this token valid at time T in the past?", the tooling and the data are not very helpful.

ACDCs are signed, but it would be more accurate to say that ACDCs are signed by an *identifier*, not just a *key*. (The key used to sign a JWT can belong to an identifier, but the `kid` field points at the key, not the identifier. The indirection in ACDCs is crucial, because it allows keys to be rotated without invalidating any historical signatures. It also allows an ACDC to be signed by a multisig group, not just by a single key.)

The signature on an ACDC is outside the data structure. It is embodied in a signed event associated with data structures called a TEL (transaction event log) and KEL (key event log). These data structures offer many features that have been touted for blockchains, including built-in, realtime revocation support and monotonic, tamper-evident timestamping (a much stronger guarantee than `iat` in JWT). The events in KELs and TELs do not contain the signed data of an ACDC — only a cryptographic commitment to it. Thus, they are privacy-preserving. Also, they are not giant, global, shared data structures — they are tiny, standalone files specific to a single identifier. The KEL and TEL of one identifier typically have different storage and infrastructure from the KELs and TELs of other identifiers, so they have none of the scale and performance bottlenecks, none of the centralized governance challenges, and none of the regulatory issues with data locality and right to be forgotten that are associated with a blockchain.

KELs are notarized by publicly transparent, independent witnesses. Malicious witnesses can't fake the data they notarize. Witnesses can guarantee the detection of malicious signers who attempt to fork their key state or signing history. Arbitrary observers can poll or subscribe to witnesses to compile their own records of whatever subsets of evidence are interesting.

The overall effect of these design choices is that, although signatures on SD-JWTs and signatures on ACDCs are equivalent in terms of pure cryptographic theory, ACDC are much stronger as practical evidence. They can be created by groups of people satisfying sophisticated trust policies. Their binding effect is stable across changes in those policies, across key rotations, and even across changes in cryptography (e.g., to upgrade for quantum resistance). They are highly available, published by parties with multiple loci of control. They can be analyzed from arbitrary standpoints in time. They require no central governance or coordination.

2.4 Chaining

With SD-JWTs, the existing JWT framework provides a standard envelope; what's interesting is the data inside. This data, called claims, is selectively disclosable. Headers (`alg`, `typ`, `kid`, `jku`, `x5u`, `x5c`, `x5t`, `cty`, `crit`...) are not. A single, standalone envelope, with whatever expanded or hashed claims it contains, is the unit of signing, data exchange, and verification.

ACDCs allow graduated disclosure of everything up to and including a containing envelope. Further, the envelope has its own identifier (called a Self-Addressing Identifier, or SAID). This allows one ACDC to reference another by its SAID, which means that multiple ACDCs can be chained together to form a sophisticated, many-node graph of verifiable data. ACDCs don't have to be standalone (though they can be). Also, the data in an ACDC graph may come from multiple issuers, and have multiple schemas: *Here is evidence of X's entitlement, and here are three pieces of verifiable data, with different schemas, from other issuers, that justify that entitlement. and here are some additional pieces of evidence that establishes the origins of the verifiable data*... This is the "chaining" feature in "Authentic Chained Data Containers". The unit of data exchange and verification with ACDCs is thus any subset of a full data graph, which makes data exchange and caching more efficient, and gives much greater precision and power to graduated disclosure.

A practical consequence of the standalone envelope model is that SD-JWTs will need registries of trusted issuers. because verifiers are limited to asking whether a particular envelope is valid in isolation, with issuers coming from a trusted list. Such registries will need to be centralized, governed, and managed for scale and performance. They are a headache when crossing jurisdictional boundaries.

On the other hand, ACDCs prove the bona fides of issuers by chaining to other ACDCs. This is dynamic and efficient, and instead of positing new APIs or services or interaction models to qualify issuers, it reuses the same mechanism to qualify issuers as is used to qualify holders. The issuer landscape can evolve minute-by-minute, and react to revocations anywhere in a chain, without any central coordination.

Thus, SD-JWTs are likely to be relevant only within the context of whatever ecosystem governs and operates the registries that justify trust in their particular issuers. This will reinforce boundaries between verticals — SD-JWTs that are relevant to supply chain will probably not be relevant as evidence in banking or telecom. On the other hand, ACDCs can cross vertical boundaries easily and dynamically, since no registries and no prior consensus on roots of trust is required.

2.5 Serialization details

SD-JWTs are pure JSON. Cryptographic primitives are child JSON objects (e.g., JWS, JWK). In their more familiar compact form, the JSON of an SD-JWT will be transformed into three base64url sections, separated by dots. This makes the format less human-friendly, but there is good tooling that converts between the two forms, and many developers are already familiar with the conventions.

ACDCs are rendered in CESR. CESR is a serialization format that has both a text and a binary representation. A signature is generated for only one representation (normally, text), but since the representations are isomorphic, it can be tested against the data regardless of which representation one receives. That is, a single signature provides verifiability for all representations. The text representation for CESR is JSON with a few additional rules. For example, cryptographic primitives in CESR are self-describing strings, not JSON subobjects. This makes a text version of CESR terser and less nested than the JSON form of a JWT, or even of CBOR. CESR also has two additional compaction strategies. The first is to transform from text to binary, which is significantly smaller than even a compact JWT. More interesting, because of graduated disclosure, ACDCs can elide data that's already known from another context, allowing compaction down to something as small as a single identifier (hash) of about 44 bytes for an entire data structure of arbitrary size (multiple KB), or an entire tree of data structures (possibly, MB). Binary ACDCs with redundancies elided are maximally efficient in terms of bandwidth and storage space.

CESR libraries with only very simple dependencies are available in python, rust, C#, javascript, elixir, and other languages. However, they are not as mature or as well known as JOSE tools used for JWTs.

2.6 Intellectual and business roots

The inventors of SD-JWTs include the inventors of JWTs, plus people from Microsoft and Ping Identity. Their thinking is closely affiliated with OpenID Connect and OAuth2. These are smart people who are well connected to big tech.

ACDCs were invented by Sam Smith, with collaboration from Phil Feairheller and others in the Trust Over IP and Web of Trust communities. Trust Over IP is an initiative of the Linux Foundation. Sam is a co-inventor of Sovrin with significant experience at Consensys and in other cybersecurity and blockchain-related initiatives. He also wrote the RAET protocol that was a key component of SaltStack. Via their sponsorship by GLEIF, which was in turn commissioned by the G20's Regulatory Oversight Committee, ACDCs are more connected to global banking and less connected to big tech or to classic identity technologies.

2.7 Standards path

SD-JWTs are being standardized at IETF, where JWT, JWS, and JWK were standardized years ago. At least two different draft RFCs are involved, and their maturity differs. They are not standards yet, but they do have sponsoring workgroups, and they are likely to become public standards-oriented RFCs at some point in the future. Implementations are very young, but Microsoft's enthusiasm seems likely to produce gravitas.

The recommended approach to schemas in SD-JWTs (standardizing collections of claims with their associated names, data types, occurrence rules, and semantics) is to register them with IANA. This is a separate step from standardizing the general technology.

ACDCs have a stable spec and several years of implementation effort. The first ACDC-based global standard was ISO17442-3 (vLEIs), which was finalized under GLEIF sponsorship in October 2024. This standard depends on ACDCs.

Schemas for ACDCs are JSON-Schema objects (an existing standard). Schemas do not require standardization — just publication and consensus on use within an ecosystem. In this sense, schema development and evolution for ACDCs is probably lighter weight than for SD-JWTs.